

lab_notebook_1_abf_intro

April 6, 2025

My Custom Title

Your Name

April 6, 2025

1 Exploring abf files

This first notebook aims to demonstrate how to analyze electrophysiological recordings from a single cell by:

- Extracting basic characteristics of the recorded signals.
- Plotting, for each recording trial (sweep, see below), the evolution of the membrane potential (voltage) and, if applicable, the injected current.

We import a few libraries:

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import pyabf                      # load the pyABF library (cf. infra)
import seaborn as sns             # used for a Kernel Density Estimate (KDE) plot
```

We focus on specific cell, the cell # 89:

```
[2]: file_path = "lhb_bursting/cell89basal.abf"
print('path to the abf file : ',file_path)
```

path to the abf file : lhb_bursting/cell89basal.abf

1.1 abf files and pyabf library

1.1.1 ABF File Overview

An ABF (Axon Binary Format) file is a proprietary file format developed by Axon Instruments (now part of Molecular Devices) to store electrophysiological data from experiments. ABF files are commonly used to save data from experiments like patch-clamp recordings, where researchers measure electrical signals from biological systems (such as neurons or muscle cells). These files can store a variety of information, including:

- Data Traces: Time series data for one or more channels, representing signals such as voltage or current.
- Metadata: Information about the experiment, including settings for the recording, such as sampling rate, experiment type, and device configuration.
- Multiple Sweeps: An ABF file can contain multiple sweeps (individual trials or experimental runs), which may differ in parameters or conditions.

The ABF format is binary, making it efficient for large datasets, but it is not easily readable without specialized software or libraries.

1.1.2 pyabf Library

The `pyabf` library is a Python package designed to facilitate working with ABF files. It provides an easy-to-use interface to read and manipulate data stored in ABF files. The library makes it simpler for researchers to extract relevant information from ABF files, without having to manually parse the binary data.

Key Features of `pyabf`:

1. Load ABF Files: Load an ABF file into memory and provide access to its data.
2. Access Data Traces: Extract time-series data, such as voltage and current traces (from ADC channels).
3. Multiple Sweep Support: Handle multiple sweeps (individual experimental runs) within a single ABF file.
4. Extract Metadata: Retrieve metadata like channel names, experiment parameters, and other settings.
5. Sweep Navigation: Select and navigate through multiple sweeps (trials) and analyze their data individually.

Common Functions in `pyabf`:

- `ABF(file_path)`: Initializes an ABF object from a given file path, loading the data into memory.
- `setSweep(sweep_index)`: Selects a specific sweep (experimental run) by its index.
- `sweepY`: Extracts the voltage (or other signal) data for the current sweep.
- `sweepX`: Extracts the time vector for the current sweep.
- `sweepC`: Extracts the command input (if available) for the current sweep.
- `adcNames`: List of ADC channel names.
- `dacNames`: List of DAC channel names.

1.1.3 On the net

The `pyABF` library was created by [Scott Harden](#). Scott Harden has made `pyABF` available as an open-source library, aiming to simplify the process of working with ABF files in Python, making it easier for researchers to analyze and visualize their data. You can find more about `pyABF` and its documentation on his website: `pyABF` - A simple Python interface for Axon Binary Format ABF files.

- [pyABF - A simple Python interface for Axon Binary Format ABF files](#), with [git repository](#)
- a [tutorial](#) by [Scott W Harden]
- in Python Package Index [pypi](#)

1.2 Basics about abf objects

Where we import the `pyabf` package and load a record file:

```
[3]: abf = pyabf.ABF(file_path)           # we load it
      print(abf)                          # record characteristics
```

ABF (v2.6) with 1 channel (pA), sampled at 10.0 kHz, containing 30 sweeps, having no tags, with a total length of 11.11 minutes, recorded without a protocol file.

Here `abf = pyabf.ABF(file_path)` creates an `abf` object that have: - **attributes**: data stored in the object, and - **methods**: functions that belong to an object and can be called to perform actions

1.2.1 Attributes

We can print more attributes:

```
[4]: print(f"{'File Path:':>20} {abf.abfFilePath}")
      print(f"{'File Version:':>20} {abf.abfVersionString}")
      print(f"{'Sampling Rate:':>20} {abf.dataRate} Hz")
      print(f"{'Total Sweeps:':>20} {abf.sweepCount}")
      print(f"{'ADC Channels:':>20} {abf.adcNames}")
      print(f"{'DAC Channels:':>20} {abf.dacNames}")
      print(f"{'Channel Units:':>20} {abf.sweepUnitsY}")
      print(f"{'Experiment Date:':>20} {abf.abfDateTime}")
```

```
File Path: /Users/campillo/Documents/1-now/_git.nosync/pepyna/data/lhb
_bursting/cell89basal.abf
File Version: 2.6.0.0
Sampling Rate: 10000 Hz
Total Sweeps: 30
ADC Channels: ['Waveform']
DAC Channels: ['AO #0']
Channel Units: pA
Experiment Date: 2024-11-08 00:00:50.086000
```

1.2.2 Methods

You can list all the methods of an `abf` object with `print(abf.__dict__)`.

```
[5]: methods = [method for method in dir(abf) if callable(getattr(abf, method)) and
               ↪ not method.startswith("__")]
      print("\n".join(methods))
```

```
_dtype
_getAdcNameAndUnits
_getDacNameAndUnits
_id_helper
_loadAndScaleData
_makeAdditionalVariables
_readHeadersV1
_readHeadersV2
getAllXs
getAllYs
headerLaunch
```

```
launchInClampFit
saveABF1
setSweep
sweepD
```

You have private methods (Prefixed with `_`), and:

- `getAllXs()`: Returns all time points (X-values) for every sweep, useful for plotting.
- `getAllYs()`: Returns all recorded signal values (Y-values) for every sweep.
- `headerLaunch()`: Likely a utility function for debugging or inspecting header information.
- `launchInClampFit()`: Opens the ABF file in ClampFit, a software from Molecular Devices used for electrophysiology data analysis.
- `saveABF1()`: Converts and saves the ABF file in version 1 format, which is older but sometimes required for compatibility.
- `setSweep(sweepIndex)`: Sets the current sweep (i.e., trial or recording segment) to a given index for further processing.
- `sweepD`: Likely an attribute or method that provides the time duration of a sweep.

Of course the main parts of the sweep are the **recorded signal** and the **command input**:

```
[6]: # Print voltage trace (recorded signal)
print(f'Voltage Trace (mV):':>25} {abf.sweepY}")

# Print command input (if available)
print(f'Command Input (mV):':>25} {abf.sweepC}")
```

```
Voltage Trace (mV): [-61.2   -60.856 -61.2   ... -56.228 -56.196 -56.353]
Command Input (mV): [0.  0.  0.  ... 0.  0.  0.]
```

1.3 Basic abf file exploration

1.3.1 Main abf attributes and methods

The `abf` object contains various attributes and methods that allow you to access metadata and data from the `.abf` file. Here are some useful attributes and how to call them:

```
[7]: print("List of sweep indexes:", ", ".join(map(str, abf.sweepList)))
```

```
List of sweep indexes: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29
```

```
[8]: sweep_index = 0 # Choose a specific sweep (e.g., first sweep -> index 0)
abf.setSweep(sweep_index)

# Print voltage trace (recorded signal)
print(f'Voltage Trace (mV):':>25} {abf.sweepY}")

# Print command input (if available)
print(f'Command Input (mV):':>25} {abf.sweepC}")

# Check all available ADC channels (recorded signals)
```

```
print(f"{'Recorded Channels:':>25} {abf.adcNames}")

# Check DAC channels (command input signals)
print(f"{'Command Channels:':>25} {abf.dacNames}")
```

Voltage Trace (mV): [-61.2 -60.856 -61.2 ... -56.228 -56.196 -56.353]
Command Input (mV): [0. 0. 0. ... 0. 0. 0.]
Recorded Channels: ['Waveform']
Command Channels: ['AO #0']

1.3.2 Some statistics of 1 sweep

Here, we have selected `sweep_index = 0`, representing the first sweep in the ABF file. We then compute some basic statistics of the corresponding voltage trace, such as the mean, median, min, max, standard deviation, and range of the signal:

```
[9]: data = abf.sweepY # The voltage trace for the specific sweep

stats = {
    "Mean (mV)": np.mean(data),
    "Median (mV)": np.median(data),
    "Min (mV)": np.min(data),
    "Max (mV)": np.max(data),
    "Std Dev (mV)": np.std(data),
    "Range (mV)": np.ptp(data), # Max - Min
}

print("\nVoltage Trace Statistics:")
for key, value in stats.items():
    print(f"{'key':>20}: {value:.3f}")
```

Voltage Trace Statistics:
Mean (mV): -55.656
Median (mV): -56.853
Min (mV): -66.204
Max (mV): 49.191
Std Dev (mV): 9.172
Range (mV): 115.395

1.4 Basics plottings

1.4.1 Plotting the voltage trace distribution

```
[10]: # Create a figure with two subplots (1 row, 2 columns)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Plot histogram on the first subplot
axes[0].hist(data, bins=50, edgecolor='black', alpha=0.7)
```

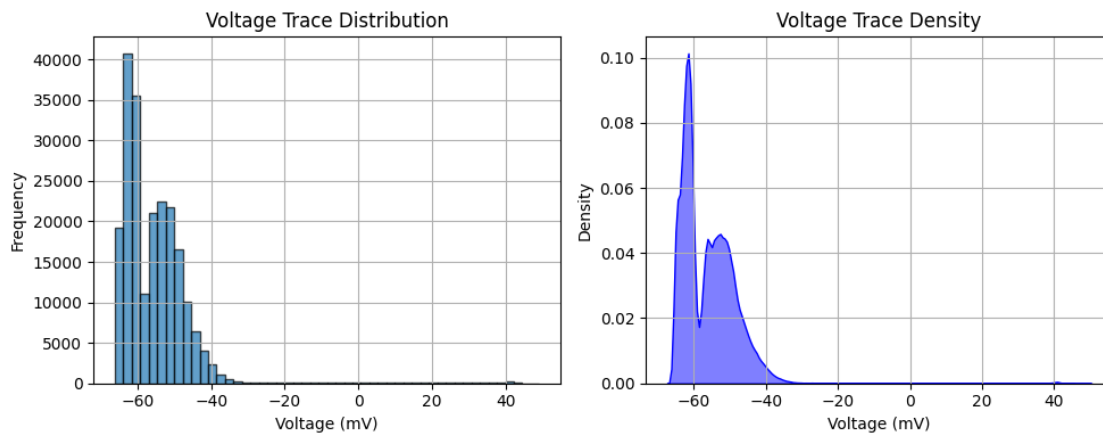
```

axes[0].set_xlabel("Voltage (mV)")
axes[0].set_ylabel("Frequency")
axes[0].set_title("Voltage Trace Distribution")
axes[0].grid(True)

# Plot KDE on the second subplot
sns.kdeplot(data, bw_adjust=0.5, fill=True, color="b", alpha=0.5, ax=axes[1])
axes[1].set_xlabel("Voltage (mV)")
axes[1].set_ylabel("Density")
axes[1].set_title("Voltage Trace Density")
axes[1].grid(True)

# Adjust layout
plt.tight_layout()
plt.show()

```



Of course, the previous plots are not particularly interesting, as they represent the distribution of the signal while mixing both inter-burst and burst phases. A more insightful approach would be to plot the distributions separately: one for the inter-burst signal and another for the intra-burst signal. Note that the previous plots exhibit a bimodal distribution, which is certainly related to the two phases of the signal—burst and non-burst (quiescent or background activity).

1.4.2 Plotting all the sweeps of an abf file

```

[11]: def plot_abf():
    plt.figure(figsize=(8, 5))

    # plot every sweep (with vertical offset)
    for sweepNumber in abf.sweepList:
        abf.setSweep(sweepNumber)
        offset = 140*sweepNumber
        plt.plot(abf.sweepX, abf.sweepY+offset, color='b',lw=0.1)

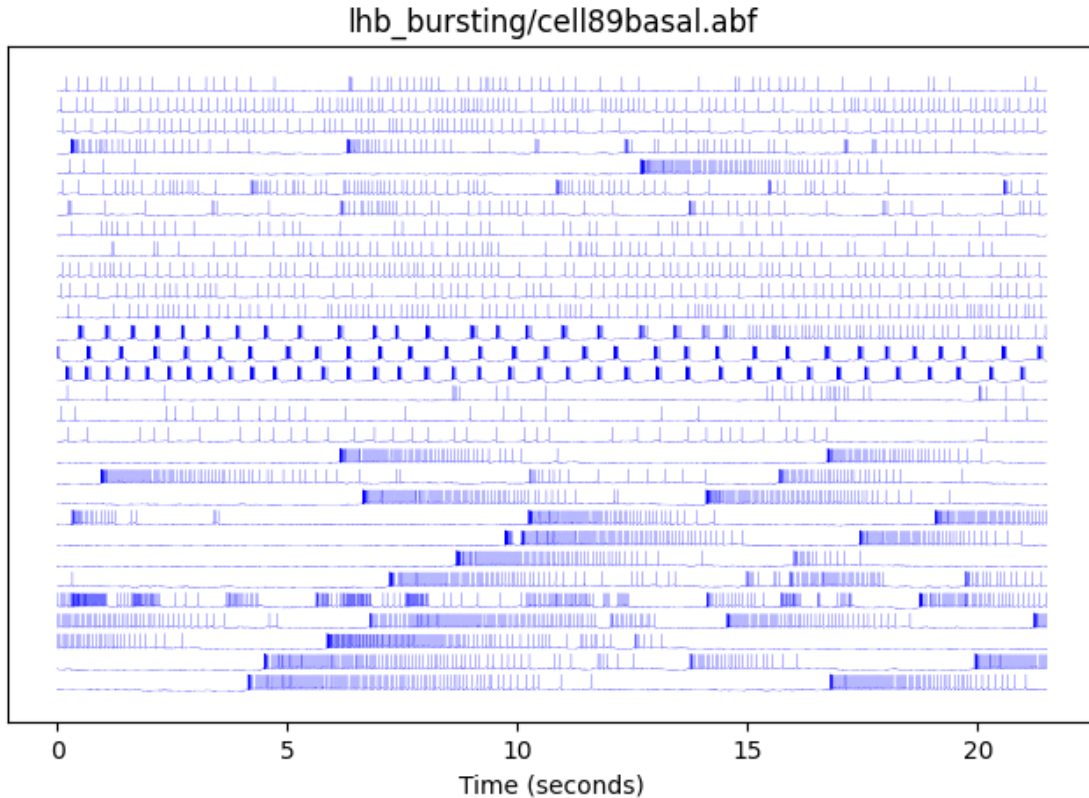
```

```

# decorate the plot
plt.gca().get_yaxis().set_visible(False) # hide Y axis
plt.title(file_path)
plt.xlabel(abf.sweepLabelX)
plt.show()

```

```
[12]: plot_abf()
```



The same but we also plot the input voltage on each sweep: here it's not so interesting because they are null (not recorded ?):

```

[13]: def plot_abf():
    plt.figure(figsize=(8, 5))

    # Plot every sweep with vertical offset
    for sweepNumber in abf.sweepList:
        abf.setSweep(sweepNumber)
        offset = 140 * sweepNumber
        plt.plot(abf.sweepX, abf.sweepY + offset, color='b', lw=0.1) #
    ↪Recorded voltage

```



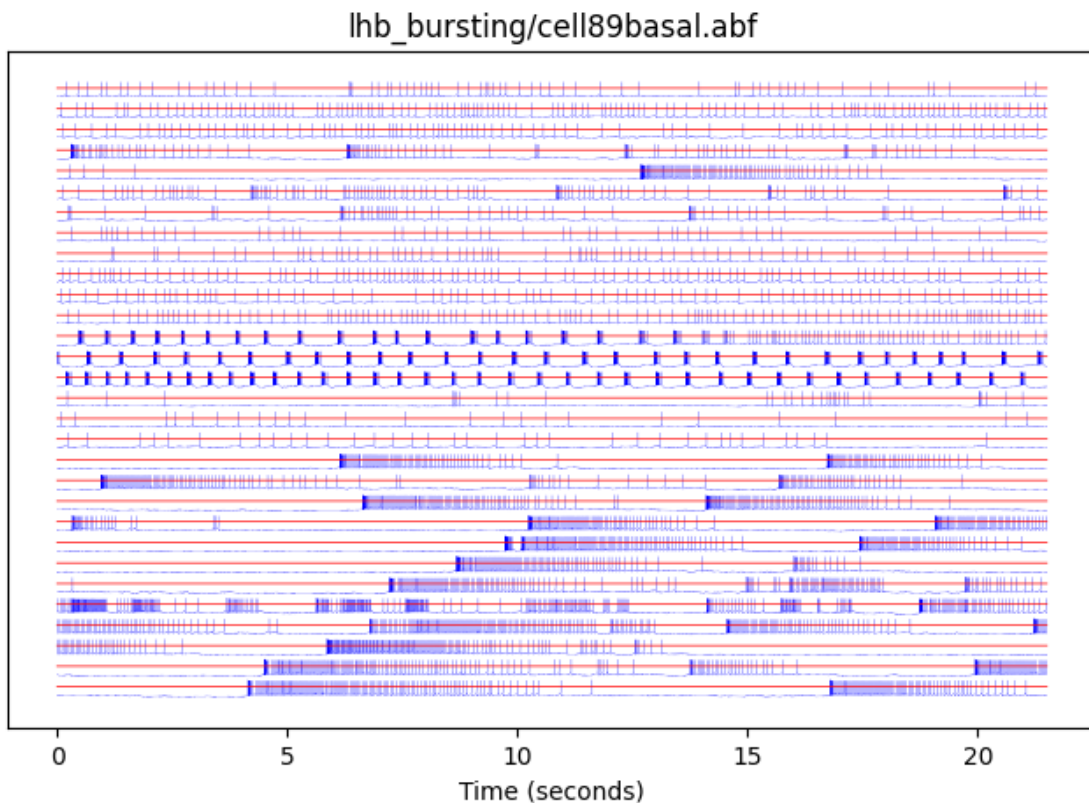
```

plt.plot(abf.sweepX, abf.sweepC + offset, color='r', lw=0.5) # Command
↪ waveform in red

# Decorate the plot
plt.gca().get_yaxis().set_visible(False) # Hide Y axis
plt.title(file_path)
plt.xlabel(abf.sweepLabelX)
plt.show()

plot_abf()

```



1.4.3 Plotting a single sweep

```

[14]: # Create the figure
fig, ax1 = plt.subplots(figsize=(8, 5))

# Plot the recorded curve (ADC) on the left axis
ax1.plot(abf.sweepX, abf.sweepY, color='b', lw=0.2, label="ADC waveform")
ax1.set_xlabel(abf.sweepLabelX)
ax1.set_ylabel(abf.sweepLabelY, color='b')
ax1.tick_params(axis='y', labelcolor='b')

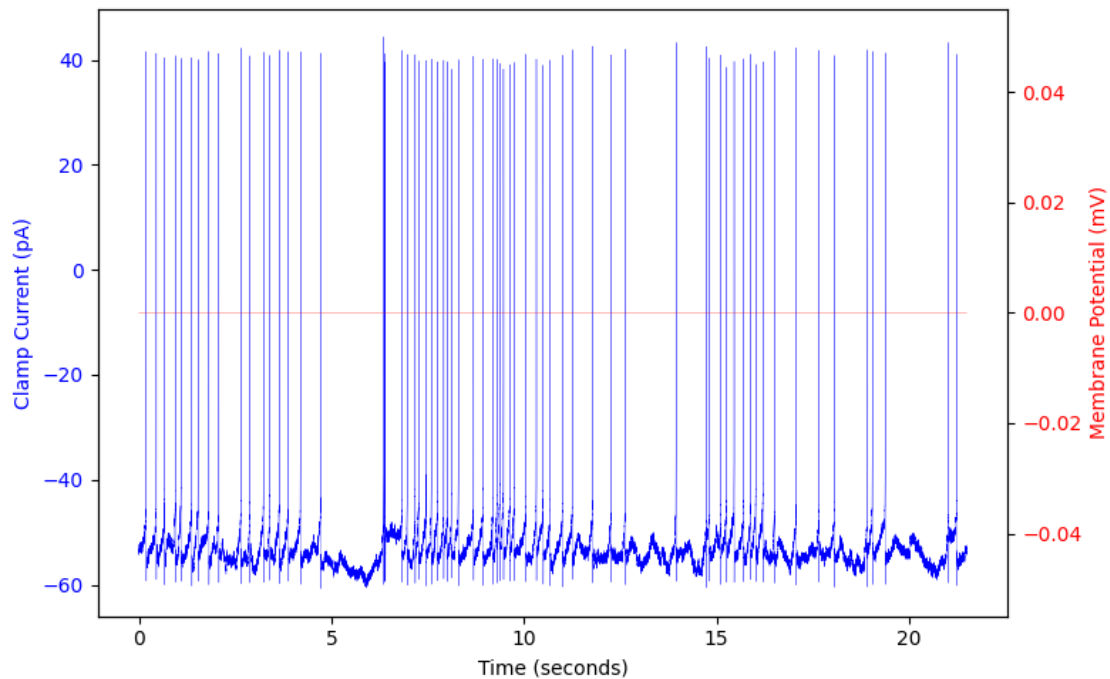
```

```

# Create a second y-axis for the control curve (DAC)
ax2 = ax1.twinx()
ax2.plot(abf.sweepX, abf.sweepC, color='r', lw=0.2, label="DAC waveform")
ax2.set_ylabel(abf.sweepLabelC, color='r')
ax2.tick_params(axis='y', labelcolor='r')

# Improve the layout
fig.tight_layout()
plt.show()

```



1.5 Data to explore

We define a list of tuples called `data_to_explore`. Each element in `data_to_explore` is a tuple containing: - a string = the file path of an `.abf` file, - An integer = a sweep number associated with the file,

representing the cell records and the specific number of the sweep we want to explore.

```

[15]: data_to_explore = []
data_to_explore.append(("lhb_bursting/cell121basal.abf", 5))
data_to_explore.append(("lhb_bursting/cell189basal.abf", 15))
data_to_explore.append(("lhb_bursting/cell191basal.abf", 4))
data_to_explore.append(("lhb_bursting/cell198basal.abf", 9))
data_to_explore.append(("lhb_bursting/cell1104basal.abf", 10))

```

```

data_to_explore.append(("lhb_bursting/cell105basal.abf", 7))
data_to_explore.append(("lhb_bursting/cell107basal.abf", 0))
data_to_explore.append(("lhb_bursting/cell209basal.abf", 4))

for abf_file, sweep_number in data_to_explore:
    print(f"ABF File: {abf_file:<30} {'Sweep Number:':>15} {sweep_number:>3}")

```

```

ABF File: lhb_bursting/cell21basal.abf      Sweep Number: 5
ABF File: lhb_bursting/cell89basal.abf      Sweep Number: 15
ABF File: lhb_bursting/cell91basal.abf      Sweep Number: 4
ABF File: lhb_bursting/cell98basal.abf      Sweep Number: 9
ABF File: lhb_bursting/cell104basal.abf     Sweep Number: 10
ABF File: lhb_bursting/cell105basal.abf     Sweep Number: 7
ABF File: lhb_bursting/cell107basal.abf     Sweep Number: 0
ABF File: lhb_bursting/cell209basal.abf     Sweep Number: 4

```

```

[16]: # Créer la figure
plt.figure(figsize=(10, 6))

# Tracer chaque fichier/sweep avec un décalage vertical
for i, (file_path, sweep_num) in enumerate(data_to_explore):
    abf = pyabf.ABF(file_path)
    abf.setSweep(sweep_num)
    offset = i * 150 # décalage vertical entre les tracés
    plt.plot(abf.sweepX, abf.sweepY + offset, lw=0.3, label=f"{file_path}_
    ↳[sweep {sweep_num}]")

# Mise en forme
plt.gca().get_yaxis().set_visible(False) # cacher l'axe Y
plt.xlabel("Temps (s)")
plt.title("Tracés en tension pour différents fichiers/sweeps")
plt.legend(fontsize="xx-small", loc="upper right")
plt.tight_layout()
plt.show()

```

Tracés en tension pour différents fichiers/sweeps

